CODING PLATFORM / STUDENT PROGRESS TRACKER

A OOPS Through C++ Cornerstone Project Report submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

GADI V V LALITHA JAHNAVI – 25B11AI586

NUJELLA PADMINI – 25B11AI854

PATNALA MEENAMRUTHA SRI NUKAMBIKA – 25B11AI914

ALLAKA JYOTHSNA – 25B11AI027

Under the esteemed guidance of

Dr. Maganti Venkatesh

Associate Professor



ADITYA

UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY
Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the OOPS Through C++ Cornerstone Project report entitled "**CODING PLATFORM / STUDENT PROGRESS TRACKER**" is being submitted by GADI V V LALITHA JAHNAVI (Roll No. 25B11AI586), NUJELLA PADMINI (Roll No. 25B11AI854), PATNALA MEENAMRUTHA SRI NUKAMBIKA (Roll No. 25B11AI914) and ALLAKA JYOTHSNA (Roll No. 25B11AI027) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Cornerstone Project** for the course **OOPS Through C++ (2501CS08)** during the academic year **2026–2027**, and records the Source Code implementation with coding standard practices and documentation undertaken by the team.

Project Guide

Dr. Maganti Venkatesh
Associate Professor

Head of the Department

Dr. K. Naga Bhargavi,
Associate Professor

External Examiner

DECLARATION

We hereby declare that the OOPS Through C++ Cornerstone Project entitled "**CODING PLATFORM / STUDENT PROGRESS TRACKER**" is a genuine work carried out by us. The Source Code implementation with coding standard practices and documentation presented in this report are our own work, completed under the guidance of **Dr. Maganti Venkatesh**, Associate Professor, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

GADI V V LALITHA JAHNAVI (25B11AI586)

NUJELLA PADMINI (25B11AI854)

PATNALA MEENAMRUTHA SRI NUKAMBIKA (25B11AI914)

ALLAKA JYOTHSNA (25B11AI027)

Date: _____

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide **Dr. Maganti Venkatesh**, Associate Professor, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to **Dr. K Naga Bhargavi**, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We also extend our sincere thanks to **Dr. M V RAJESH,** Associate Dean, School of Computing, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh – Registrar, Dr. A. Ramesh - Pro VC (E&S), Dr. S. Rama sree - Pro VC (Academics), Dr. M B Srinivas – Vice Chancellor and Dr. M. Sreenivasa Reddy - Dy. Pro-Chancellor and Management, Aditya university**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank our **Teaching and Non-Teaching Staff, Parents and Friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Project Associates

GADI V V LALITHA JAHNAVI 25B11AI586

NUJELLA PADMINI 25B11AI854

PATNALA MEENAMRUTHA SRI NUKAMBIKA 25B11AI914

ALLAKA JYOTHSNA 25B11AI027

ABSTRACT

Students today practise programming on several independent platforms, so assigned problems, submissions and progress are scattered and faculty find it slow to monitor them. This project, Coding Platform / Student Progress Tracker, is a menu-driven console application developed in C++ to manage coding activity within an academic department. It provides three separate roles: Admin, Faculty and Student. The design uses the classes Person, Student, Faculty, Platform, Problem, StudentPlatform, Assignment and Submission, and demonstrates encapsulation, inheritance, abstraction, a virtual display function, function overloading and a friend function. Records are kept in STL vectors, ranked, searched and sorted with standard algorithms, checked by input-validation functions and exception handling, and stored in text files using ifstream and ofstream. Faculty add problems from LeetCode, CodeChef, HackerRank, GeeksforGeeks and Codeforces, assign them with deadlines and review submissions, while students link profile addresses, record submissions and view progress. The application stores links and statuses entered by users and does not contact external platforms. The outcome is a working centralised tracker that replaces scattered manual records.

INDEX

Chapter	Title	Page No.
	Title Page	I
	Certificate	II
	Declaration	III
	Acknowledgement	IV
	Abstract	V
	Index	VI
	Abbreviations	VII
	List of Figures	VIII
	List of Tables	IX
1	**Introduction**	1
	1.1. Introduction to the Domain	1
	1.2. Problem Statement	1
	1.3. Project Objectives	2
	1.4. Planned Modules	2
	1.5. System Architecture	4
	1.6. Algorithm and Flowchart	5
	1.7. System Requirements	8
	1.8. Project Planning and Team Contributions	8
2	**Project Implementation**	10
	2.1. OOP Concepts Used for Implementation	10
	2.2. Class Design and Relationships	16
	2.3. Implementation Details	20
	2.4. Code	24
	2.5. Output Screens	27
3	**Conclusion & Future Scope**	30
	3.1. Conclusion	30
	3.2. Future Scope	30

ABBREVIATIONS

Abbreviation	Full Form
B.Tech	Bachelor of Technology
CGPA	Cumulative Grade Point Average
ID	Identifier
OOP	Object-Oriented Programming
OOPS	Object-Oriented Programming System
STL	Standard Template Library
URL	Uniform Resource Locator

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
1.1	System architecture of the Coding Platform / Student Progress Tracker	5
1.2	Program execution flow from start-up to exit	6
1.3	Submission recording, review and progress workflow	7
2.1	Class hierarchy: Person, Student, Faculty and Timetable	17
2.2	Platform, Problem, StudentPlatform, Assignment, Submission and ProgressStats	18
2.3	ID-based relationships between the principal classes	19
2.4	Main menu and rejected administrator login (Admin Login with an incorrect password)	27
2.5	Admin menu, option 2: students ranked by CGPA (first ten of twenty sample records)	28
2.6	Faculty menu, option 6: supported coding platforms	28
2.7	Faculty menu, option 14: progress of student 25B11AI586	28
2.8	Student menu, option 4: assigned problems with assignment state (student 25B11AI586)	29
2.9	Student menu, option 8: student summary and progress	29

LIST OF TABLES

Table No.	Title of the Table	Page No.
1.1	Coding platforms represented in the program	3
1.2	Planned modules and their implementation status	3
1.3	Presentation items not implemented in the submitted source code	4
1.4	Software and hardware requirements	8
1.5	Project plan stated in the presentation	8
1.6	Team members and stated responsibilities	9
2.1	OOP concepts and where they appear in the program	10
2.2	Summary of the classes and the structure	18
2.3	Options of the Admin menu (adminMenu)	20
2.4	Options of the Faculty menu (facultyMenu)	21
2.5	Options of the Student menu (studentMenu)	22
2.6	Validation functions	22
2.7	Assignment states and the progress counters	23
2.8	Files created and used by the program	23

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

The domain of this project is **coding activity management and student progress tracking** in an academic department. Programming practice is now a regular part of engineering education, and students solve problems on public coding platforms such as LeetCode, CodeChef, HackerRank, GeeksforGeeks and Codeforces. Faculty members choose problems for their classes, set deadlines and expect to know who has completed the work. Each platform, however, keeps its own record of a student's activity, and none of them knows which problems a particular faculty member assigned or when they are due.

On a normal day the department handles several kinds of information. For every student it holds an identification number, name, e-mail address, year of study, department, timetable number and CGPA. For every faculty member it holds an identification number, subject and course code. The coding work itself consists of problems (title, difficulty, topic, platform and a direct link), assignments that connect a problem to a student with a deadline, and submissions that record the link to the student's solution, the date and a review status. From these records a faculty member needs to see how many problems each student has completed, which are waiting for review and which are overdue.

This project implements that workflow as an **object-oriented C++ console application**. It is not a database system and it is not a general student-management program: its purpose is to assign coding problems, record submissions and calculate progress. The application is organised around three roles, Admin, Faculty and Student, each with its own menu, and it keeps all of its data in a small set of text files so that records survive between runs.

1.2 Problem Statement

Students practise programming on multiple independent platforms. As a result, the problems assigned to them, their submissions and their overall progress remain spread across different accounts, with no single view of the work. Faculty members who assign problems must collect this information manually, usually from spreadsheets, messages or memory, which is slow, repetitive and prone to error. It is difficult to confirm that a problem was really assigned, to know whether a submission has been reviewed, or to compare the progress of students in a batch.

There is therefore a need for a single application in which administrators maintain the basic records, faculty assign problems and review submissions, and students record their own work and see their progress. The application must also reject invalid data (for example an incorrect e-mail address, an out-of-range CGPA or a problem link that does not belong to the chosen platform) so that the records remain reliable.

1.3 Project Objectives

The objectives of the project, taken from the project presentation and checked against the implementation, are:

- To develop a centralised, menu-driven application for managing coding activity and student progress.
- To maintain student, faculty and coding-platform records with validation of every input.
- To store coding problems from LeetCode, CodeChef, HackerRank, GeeksforGeeks and Codeforces together with a validated problem link.
- To allow faculty to assign a problem to one student, or to every student of a department, with a deadline.
- To allow students to record submissions and to allow only the assigning faculty member to review them.
- To calculate the progress of each student (assigned, accepted, under review and overdue problems) and to generate a ranked text report.
- To reduce manual, spreadsheet-based record keeping.
- To demonstrate the core OOP concepts of C++ (classes, encapsulation, inheritance, abstraction, polymorphism, overloading, friend functions, exception handling and file handling) in a realistic application.

1.4 Planned Modules

The presentation divides the system into six modules. Table 1.1 lists the coding platforms covered, and Table 1.2 compares each planned module with what the submitted source code actually implements. Where the two differ, the source code is treated as authoritative.

Table 1.1: Coding platforms represented in the program

Platform ID	Platform	Website stored in the Platform object
1	LeetCode	https://leetcode.com
2	CodeChef	https://www.codechef.com
3	HackerRank	https://www.hackerrank.com
4	GeeksforGeeks	https://www.geeksforgeeks.org
5	Codeforces	https://codeforces.com

These five platforms are loaded as default records by `seedPlatforms()` when the program runs for the first time; a faculty member can add further platforms. The platform website is used only to check that a problem link or profile link begins with the correct address. The application does not connect to any platform, does not verify a solution online and does not retrieve live statistics.

Table 1.2: Planned modules and their implementation status

Module	Planned in the presentation	Implemented in the source code
Admin management	Add, update and delete students and faculty; manage platforms and problems.	Add, view, update, delete, search and sort students; add, view and delete faculty; view and delete platforms; view all problems. There is no faculty-update option, and problems are added and deleted from the faculty menu.
Faculty management	Add problems, assign them, view submissions and progress, generate reports.	Add, search and delete problems; add platforms; assign a problem to a student or to a department; view own assignments; view pending and all submissions; review submissions; view student progress; generate `student_report.txt`.
Student management	Log in, view assigned problems, update status, record submissions, view progress.	View profile and CGPA rank; link platform accounts; view assigned problems with their state; search problems; record and view submissions; view progress. Students cannot mark their own work Accepted.
Problem management	Store ID, title, platform, topic, difficulty and status.	`Problem` class stores ID, title, difficulty, platform ID, topic and link. A separate status is not stored in `Problem`; it is derived for each assignment.
Submission management	Record student, problem, platform, submission count and result.	`Submission` class stores ID, student, problem, platform, link, date and status (Pending, In Progress, Accepted, Rejected). Result codes such as Wrong Answer or TLE and a submission counter are not stored.
Progress tracking	Assigned, solved and pending counts and success rate.	`ProgressStats` and `calculateProgress()` give assigned, completed, under-review and overdue counts; progress is completed divided by assigned, as a percentage.

Some items shown in the presentation are not part of the submitted source code. They are listed in Table 1.3 so that the report does not claim them as completed work.

Table 1.3: Presentation items not implemented in the submitted source code

Item in the presentation	Status in the source code
`Admin` class and `Progress` class	There is no `Admin` class; the administrator is recognised by a fixed user name and password in `main()`. Progress is calculated by the `ProgressStats` structure and `calculateProgress()` function, not by a class.
Database layer with Users, Problems, Submissions, Test Cases, Contests and Progress tables, and an ER diagram	No database is used. Data is stored in seven text files. Test cases and contests do not exist in the program. The ER-style relationships are represented in this report as relationships between classes (Figure 2.3).
`displayRole()` overridden by each role	The virtual function is named `display()`, is pure virtual in `Person`, and is overridden in `Student` and `Faculty`.
`Student::recordSubmission()`	Recording a submission is implemented as the free function `recordSubmission()`, not as a member of `Student`.
Student status values Not Solved, In Progress, Solved	Submissions use Pending, In Progress, Accepted and Rejected, and only the assigning faculty member can change them. Assignments show Completed, Under Review, OVERDUE or Not Completed.
Administrator adds and removes problems; progress dashboard	Problem addition and deletion are faculty functions. Progress is shown as console summaries and as a text report; there is no dashboard.

1.5 System Architecture

The application follows a layered structure that is typical of a console program (Figure 1.1). Users interact with a text menu; the menus call functions and member functions of the classes; the objects are held in STL vectors in memory; and the vectors are written to and read from text files. The layers are described below.

- **User layer:** the Admin, Faculty and Student roles, selected through the login menu in `main()`.
- **Console interface layer:** `main()`, `adminMenu()`, `facultyMenu()` and `studentMenu()`, which display the options and call the operations.
- **Application logic layer:** the classes `Person`, `Student`, `Faculty`, `Platform`, `Problem`, `StudentPlatform`, `Assignment`, `Submission` and `Timetable`, the `ProgressStats` structure, and the functions for validation, searching, sorting, assignment and reporting.
- **In-memory data layer:** one `vector` for every class (for example `vector<Student>` and `vector<Submission>`).
- **File storage layer:** the `load...()` functions read the data files when the program starts and the `save...()` functions write them when a role logs out and when the program exits.

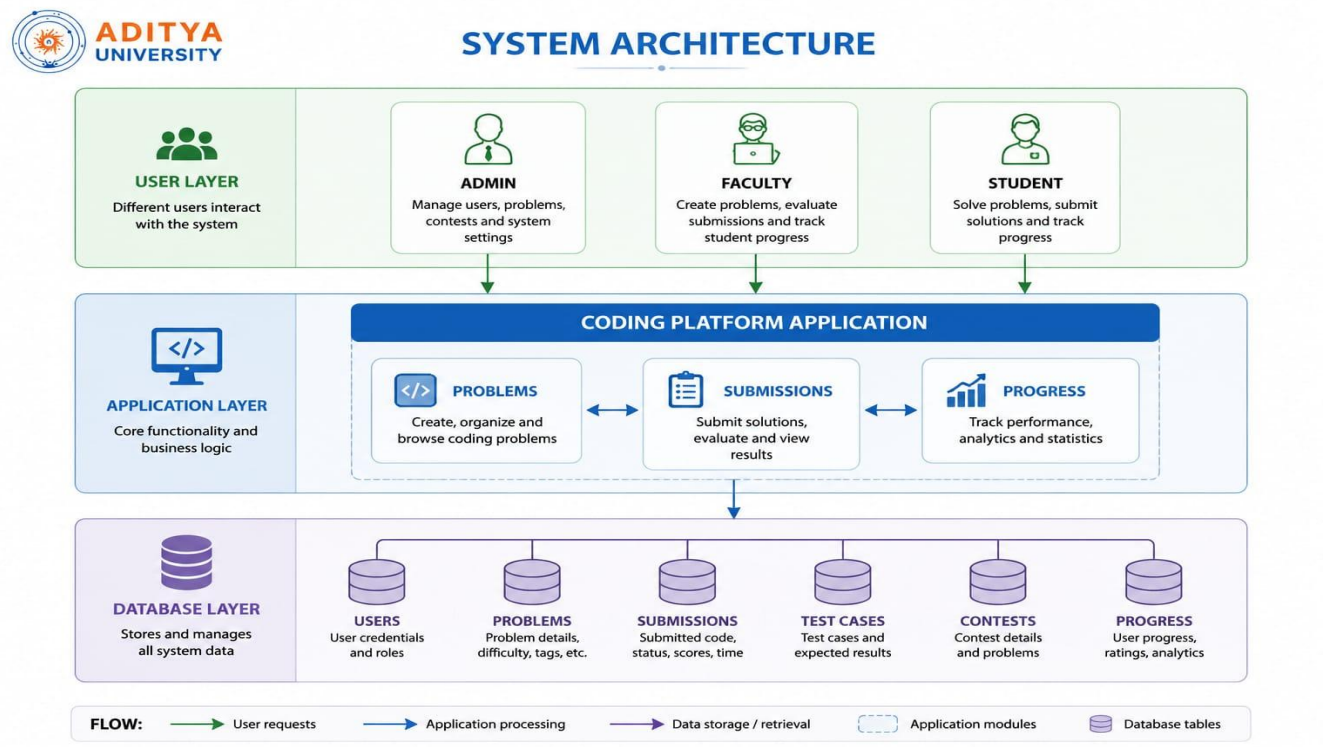


Figure 1.1: System architecture of the Coding Platform / Student Progress Tracker

The architecture slide of the presentation also shows a database layer and tables for test cases and contests. Those items are not implemented in this program; persistent storage is provided by text files, as shown in Figure 1.1.

1.6 Algorithm and Flowchart

Algorithm 1: program execution. The following steps were derived from `main()` and the three menu functions.

1. Start the program and create one vector for each class.
2. Call the `load...()` functions to read the data files. If students, faculty, problems and platforms are all empty (first run), call `seedDefaultData()`; otherwise sort the students by CGPA.
3. Display the main menu: Admin Login, Faculty Login, Student Login, View Coding Platforms, Exit.
4. For options 1 to 3, read the credentials and validate them: the fixed administrator user name and password, or an existing faculty or student ID together with the fixed role password. If the login is invalid, display a message and return to the main menu.
5. On a valid login, open the role menu (`adminMenu()`, `facultyMenu()` or `studentMenu()`).

6. Read the menu choice with `getInteger()`, perform the chosen operation (reading each value through a validating input function), update or retrieve the vectors, and display the result.
7. Repeat from step 6 until the user chooses Logout; then call `saveAllData()` and return to the main menu.
8. For option 4, display the platforms. For option 5, call `saveAllData()`, display a closing message and stop.

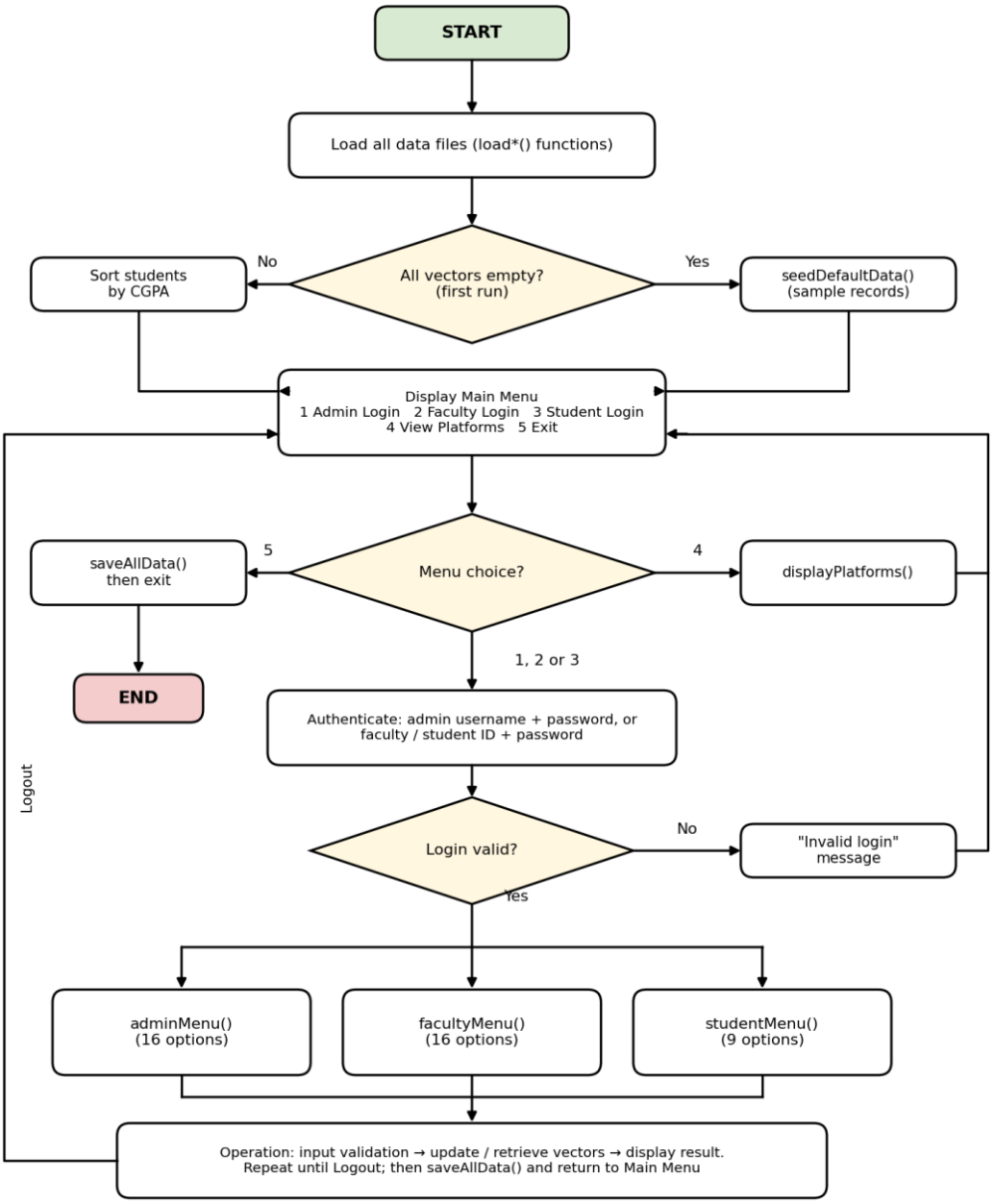


Figure 1.2: Program execution flow from start-up to exit

Algorithm 2: submission, review and progress. Figure 1.3 shows how a submission moves from the student to the faculty member and into the progress calculation. The key rules are that a student can submit only a problem that was assigned to them, that a new submission always starts as Pending, and that only the faculty member who assigned the problem may change its status.

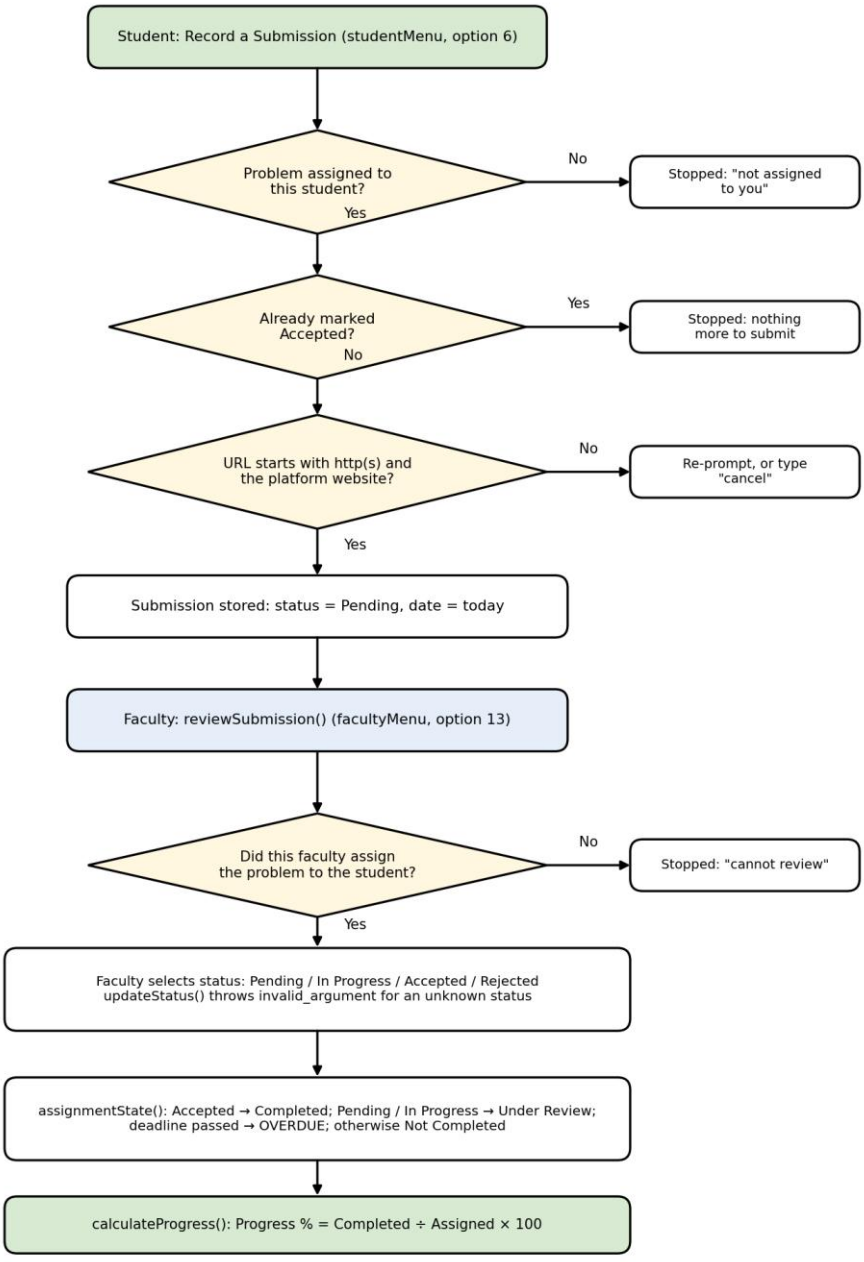


Figure 1.3: Submission recording, review and progress workflow

Algorithm 3: progress calculation (`calculateProgress()` and `studentProgress()`):

```
for each assignment belonging to the student:
    total = total + 1
    state = assignmentState(assignment)
    if state == "Completed"     then completed   = completed + 1
    else if state == "Under Review" then underReview = underReview + 1
    else if state == "OVERDUE"  then overdue     = overdue + 1
if total == 0 then display "No assignments found"
else progress % = completed / total * 100
```

1.7 System Requirements

The program uses only the C++ standard library, so no additional packages or database software are required. The requirements are given in Table 1.4. The submitted source file compiled without errors with the g++ compiler (`-std=c++11`) while this report was being prepared.

Table 1.4: Software and hardware requirements

Item	Requirement
Programming language	C++ (standard library only: iostream, string, vector, fstream, sstream, algorithm, stdexcept, iomanip, cstdlib, cctype, cmath, ctime)
Compiler	Any C++11-capable compiler, for example g++
Operating system	Any operating system that provides a text console
Input and output	Keyboard and console (text interface)
Storage	A writable working folder in which the program creates its text data files and `student_report.txt`
Hardware	No special hardware; an ordinary computer is sufficient

1.8 Project Planning and Team Contributions

The presentation describes an eight-week plan (Table 1.5) and a division of responsibilities among the four team members (Table 1.6). Both are reproduced here as the planned approach of the team, written in formal language.

Table 1.5: Project plan stated in the presentation

Week	Phase	Planned activities
1	Requirement analysis and planning	Understand the objectives, identify the Admin, Faculty and Student users, define the scope and prepare the plan.
2	System design and architecture	Design the architecture, identify classes and relationships, plan the file structures and prepare class diagrams.
3	Admin module	Administrator login; manage students, faculty and platforms; basic file handling.
4	Faculty module	Faculty login; add and manage problems; assign problems; view assigned students; validation and error handling.
5	Student module	Student login; view assigned problems; record submissions and view progress; file handling for submissions.
6	Progress and reports module	Track progress, calculate statistics, generate faculty reports and improve data consistency.

Week	Phase	Planned activities
7	Testing and debugging	Test each module and their integration, correct errors, and check file handling and data integrity.
8	Documentation and final delivery	Prepare the documentation, final testing, code clean-up, presentation and demonstration.

Table 1.6: Team members and stated responsibilities

No.	Team member	Responsibility stated in the presentation
1	GADI V V LALITHA JAHNAVI (25B11AI586)	Team lead: overall project coordination; the Person class and the administrator functions.
2	NUJELLA PADMINI (25B11AI854)	Faculty and Problem modules and file handling,testing,
3	PATNALA MEENAMRUTHA SRI NUKAMBIKA (25B11AI914)	Student and Submission modules ,documentation.
4	ALLAKA JYOTHSNA (25B11AI027)	Platform and Progress modules, and the presentation.

The members are listed in the order in which their names appear on the title slide of the presentation.

CHAPTER 2: PROJECT IMPLEMENTATION

2.1 OOP Concepts Used for Implementation

This section explains each object-oriented concept that is actually present in the source code, with the class or function in which it appears. Table 2.1 gives a summary and the following subsections give a definition, the role in the project and a short excerpt from the program.

Table 2.1: OOP concepts and where they appear in the program

Concept	Where it is used	Example
Classes and objects	Nine classes and one structure	`Student`, `Problem`, `Submission`, `ProgressStats`
Encapsulation	Private and protected data with public member functions	`Problem` (private members), `Student::updateCGPA()`
Inheritance	`Student` and `Faculty` derive from `Person`	`class Student : public Person`
Abstraction	Abstract base class with a pure virtual function	`Person::display() = 0`
Polymorphism (virtual functions)	`display()` overridden in derived classes; virtual destructor	`Student::display()`, `Faculty::display()`
Constructors and destructors	Constructors with default arguments; virtual base destructor	`Submission(...)`, `virtual ~Person()`
Function overloading	Two versions of `searchProblem()`	Search by keyword and by problem ID
Friend function	Non-member function with access to private data	`studentSummary()`
Exception handling	`throw invalid_argument` and `catch (exception &e)`	`updateYear()`, `updateStatus()`
File handling	`ifstream` and `ofstream` for seven data files and a report	`saveStudents()`, `loadStudents()`
STL vectors and algorithms	`vector` containers and `sort()` with comparison functions	`compareStudentByCGPA()`
Static members	Class-level helper functions without objects	`Timetable::isValid()`

2.1.1 Classes and Objects

Definition. A class is a user-defined type that groups data and the functions that operate on it; an object is an instance of a class.

Role in this project. The program defines `Person`, `Student`, `Faculty`, `Platform`, `Problem`, `StudentPlatform`, `Assignment`, `Submission` and `Timetable`, plus the structure `ProgressStats`. Every record entered by a user becomes an object that is stored in a vector.

The following statement, taken from `addStudent()`, creates and stores an object:

```
students.push_back(Student(name, id, email, year, department, timetableNo, cgpa));
```

How it works. In `addStudent()` a `Student` object is created from validated input and added to the `vector<Student>`; the vector is then sorted by CGPA.

2.1.2 Encapsulation

Definition. Encapsulation binds data and functions together and restricts direct access to the data.

Role in this project. The data of `Platform`, `Problem`, `StudentPlatform`, `Assignment` and `Submission` is private and is read only through getter functions. `Person`, `Student` and `Faculty` keep their data protected. Changes to a `Student` pass through the `update...()` functions, which reject invalid values.

```
class Problem
{
private:
    int problemID;
    string title;
    string difficulty;
    int platformID;
    string topic;
    string url;

public:
    Problem(int id = 0, string t = "", string d = "",
            int pid = 0, string top = "", string u = "")
    {
        problemID = id;
        title = t;
        difficulty = d;
        platformID = pid;
        topic = top;
        url = u;
    }
```

How it works. All six data members of `Problem` are private. They are set once by the constructor and read by `getID()`, `getTitle()`, `getDifficulty()`, `getPlatformID()`, `getTopic()` and `getURL()`, so other code cannot change a problem accidentally.

2.1.3 Inheritance

Definition. Inheritance allows a class to acquire the members of an existing class.

Role in this project. `Student` and `Faculty` are derived publicly from `Person`, which holds the name and e-mail address common to both. This avoids repeating these members and the e-mail functions.

```
class Student : public Person
{
protected:
    string studentID;
    int year;
    string department;
    int timetableNo;     // 1 - 9
    double cgpa;         // 0.00 - 10.00

public:
    Student(string n = "", string id = "", string e = "",
            int y = 1, string dpt = "", int tt = 1, double g = 0.0)
        : Person(n, e)
    {
        studentID = id;
        year = y;
        department = dpt;
        timetableNo = tt;
        cgpa = g;
    }
```

How it works. The `Student` constructor passes the name and e-mail to the base constructor `Person(n, e)` in its initialiser list and then initialises its own members. `Faculty` is built in the same way.

2.1.4 Abstraction

Definition. Abstraction exposes what an object does and hides how it does it; in C++ an abstract class declares at least one pure virtual function.

Role in this project. `Person` is an abstract class because `display()` is declared pure virtual. A `Person` object can never be created, and every derived class must supply its own `display()`. In addition, helper functions such as `getValidEmail()` and `recordSubmission()` hide the details of validation and storage from the menu code.

```
class Person
{
protected:
    string name;
    string email;

public:
    Person(string n = "", string e = "")
    {
        name = n;
        email = e;
    }

    virtual void display() const = 0;

    string getName() const { return name; }
    string getEmail() const { return email; }
    void setEmail(string e) { email = e; }

    virtual ~Person() {}
};
```

How it works. `display()` has no body in `Person`. The remaining members give both derived classes a common interface for name and e-mail.

2.1.5 Polymorphism and Virtual Functions

Definition. Polymorphism lets the same function name behave differently for different classes; a virtual function is resolved at run time when it is called through a base-class pointer or reference.

Role in this project. `Person::display()` is a pure virtual function that `Student` and `Faculty` override with their own output. `Person` also has a virtual destructor, which is the correct design for a base class.

```
    virtual void display() const
    {
        cout << "\n--- Student Details ---" << endl;
        cout << "Name       : " << name << endl;
        cout << "Student ID : " << studentID << endl;
        cout << "Email      : " << email << endl;
        cout << "Year       : " << year << endl;
        cout << "Department : " << department << endl;
        cout << "CGPA       : " << cgpa << endl;
```

```
        cout << "Timetable  : " << Timetable::describe(timetableNo) << endl;
    }
```

How it works. `Student::display()` prints the student record, including the timetable description. `Faculty::display()` prints the faculty record in the same way. In the submitted program these functions are called directly on `Student` and `Faculty` objects stored in separate vectors (for example `students[idx].display()`); the program does not hold `Person` pointers, so run-time dispatch through a base pointer is supported by the design but not exercised. Compile-time polymorphism is shown by function overloading in Section 2.1.7.

2.1.6 Constructors and Destructors

Definition. A constructor initialises an object when it is created; a destructor performs clean-up when it is destroyed.

Role in this project. Every class has a constructor with default arguments, so objects can be created with or without values. The only user-written destructor is `virtual ~Person() {}`. The other classes need none because they use `string` and `vector`, which release their own memory.

```
    Submission(int sub = 0, string sid = "", int pid = 0, int plid = 0,
               string url = "", string date = "", string st = "Pending")
    {
        submissionID = sub;
        studentID = sid;
        problemID = pid;
        platformID = plid;
        submissionURL = url;
        submissionDate = date;
        status = st;
    }
```

How it works. The `Submission` constructor assigns each member and defaults the status to "Pending", which is how every new submission starts.

2.1.7 Function Overloading

Definition. Function overloading allows several functions with the same name but different parameter lists.

Role in this project. `searchProblem()` has two forms: one searches the title and topic with a keyword and the other finds a problem by its integer ID. `searchProblemMenu()` calls the form that matches the type of the value entered.

```
void searchProblem(const vector<Problem> &problems,
                   const vector<Platform> &platforms, string keyword)
{
    bool found = false;
    for (int i = 0; i < (int)problems.size(); i++)
    {
        if (containsIgnoreCase(problems[i].getTitle(), keyword) ||
            containsIgnoreCase(problems[i].getTopic(), keyword))
        {
            if (!found) printProblemHeader();
            printProblemRow(problems[i], platforms);
            found = true;
        }
    }
    if (!found) cout << "\nNo problem matches '" << keyword << "'." << endl;
}
```

```
void searchProblem(const vector<Problem> &problems,
                   const vector<Platform> &platforms, int problemID)
{
    int idx = findProblemIndex(problems, problemID);
    if (idx == -1) { cout << "\nProblem not found." << endl; return; }
    problems[idx].display(platforms);
}
```

How it works. The compiler selects the version from the type of the last argument: `string keyword` or `int problemID`.

2.1.8 Friend Functions

Definition. A friend function is a non-member function that is allowed to access the private and protected members of a class.

Role in this project. `studentSummary()` is declared as a friend of `Student` and prints the summary that students see under "View My Progress".

```
    friend void studentSummary(const Student &s);
};
    // ...
void studentSummary(const Student &s)
{
    cout << "\nStudent Summary";
    cout << "\nName       : " << s.name;
    cout << "\nStudent ID : " << s.studentID;
    cout << "\nDepartment : " << s.department;
    cout << "\nYear       : " << s.year;
    cout << "\nCGPA       : " << s.cgpa;
    cout << "\nTimetable  : " << s.timetableNo << endl;
}
```

How it works. Because of the friend declaration, `studentSummary()` reads `s.name`, `s.studentID` and the other protected members directly instead of using getters.

2.1.9 Exception Handling

Definition. Exception handling reports and handles run-time errors through `throw`, `try` and `catch`.

Role in this project. Functions that change an object throw `invalid_argument` when a value is not acceptable: `Student::updateYear()`, `updateEmail()`, `updateTimetable()`, `updateCGPA()` and `Submission::updateStatus()`. The calling functions `updateStudent()` and `reviewSubmission()` catch the exception and display its message. The program uses the standard exception class and does not define its own exception classes.

```
    void updateYear(int newYear)
    {
        if (newYear < 1 || newYear > 4)
            throw invalid_argument("Year must be between 1 and 4.");
        year = newYear;
    }
    // ...
    try
    {
        if (choice == 1) submissions[idx].updateStatus("Pending");
        else if (choice == 2) submissions[idx].updateStatus("In Progress");
        else if (choice == 3) submissions[idx].updateStatus("Accepted");
        else if (choice == 4) submissions[idx].updateStatus("Rejected");
        else { cout << "Invalid choice." << endl; return; }
```

```
        cout << "\nSubmission status updated." << endl;
    }
    catch (exception &e)
    {
        cout << "\nError: " << e.what() << endl;
    }
```

How it works. If `updateStatus()` receives a status other than Pending, Accepted, Rejected or In Progress, it throws; the `catch` block prints "Error: Invalid submission status." and the menu continues. Invalid keyboard input is handled separately by loops in the `get...()` functions (Section 2.3.6).

2.1.10 File Handling

Definition. File handling reads data from and writes data to files through streams.

Role in this project. Each class has a `save` and a `load` function that use `ofstream` and `ifstream`. Fields are separated by the `|` character and each record occupies one line. `generateReport()` writes a formatted report with `ofstream`.

```
void saveStudents(const vector<Student> &students)
{
    ofstream file("students.txt");
    file << fixed << setprecision(2);
    for (int i = 0; i < (int)students.size(); i++)
    {
        file << students[i].getStudentID() << "|" << students[i].getName() << "|"
             << students[i].getEmail() << "|" << students[i].getYear() << "|"
             << students[i].getDepartment() << "|" << students[i].getTimetableNo() << "|"
             << students[i].getCGPA() << endl;
    }
}


void loadStudents(vector<Student> &students)
{
    ifstream file("students.txt");
    if (!file) return;


    string line;
    while (getline(file, line))
    {
        vector<string> f = splitLine(line, '|');
        if (f.size() < 7) continue;      // skips old-format / damaged lines


        int year = atoi(f[3].c_str());
        int timetableNo = atoi(f[5].c_str());
        double cgpa = atof(f[6].c_str());
        if (year < 1 || year > 4) year = 1;
        if (!Timetable::isValid(timetableNo)) timetableNo = 1;
        if (cgpa < 0.0 || cgpa > 10.0) cgpa = 0.0;


        students.push_back(Student(f[1], f[0], f[2], year, f[4], timetableNo, cgpa));
    }
}
```

How it works. `saveStudents()` writes one line per student. `loadStudents()` reads each line with `getline()`, splits it with `splitLine()`, skips damaged lines, corrects out-of-range values and rebuilds the `Student` objects.

2.1.11 STL Vectors and Algorithms

Definition. The Standard Template Library supplies container classes such as `vector` and algorithms such as `sort()`.

Role in this project. Every collection of records is a `vector`. Records are added with `push_back()`, removed with `erase()` and traversed with indexed loops. `sort()` orders students by CGPA, name or department using comparison functions, and `getStudentRank()` gives students with equal CGPA the same rank.

```
bool compareStudentByCGPA(const Student &a, const Student &b)
{
    if (fabs(a.getCGPA() - b.getCGPA()) > 0.0001) return a.getCGPA() > b.getCGPA();
    return a.getName() < b.getName();
}
    // ...
// Students with the same CGPA share the same rank.
int getStudentRank(const vector<Student> &students, const string &id)
{
    int idx = findStudentIndex(students, id);
    if (idx == -1) return -1;

    int rank = 1;
    for (int i = 0; i < (int)students.size(); i++)
        if (students[i].getCGPA() > students[idx].getCGPA() + 0.0001) rank++;
    return rank;
}
```

How it works. `compareStudentByCGPA()` returns true when the first student has the higher CGPA, and uses the name to break ties. `getStudentRank()` counts how many students have a strictly higher CGPA.

2.1.12 Static Members

Definition. A static member function belongs to the class and can be called without creating an object.

Role in this project. `Timetable` represents the nine timetables of the university. All of its members are static, so the program calls `Timetable::isValid()`, `timings()`, `labDay()`, `describe()` and `printAll()` directly.

```
class Timetable
{
public:
    enum { COUNT = 9 };

    static bool isValid(int no) { return no >= 1 && no <= COUNT; }
    // ...
    static string describe(int no)
    {
        if (!isValid(no)) return "Not assigned";
        return "Timetable " + intToString(no) + " (" + timings(no) +
               ", Lab: " + labDay(no) + ")";
    }
```

How it works. The constant `COUNT` is an unnamed enumeration value, which is used both for validation (1 to 9) and for the size of the timing tables.

2.2 Class Design and Relationships

Figure 2.1 shows the inheritance hierarchy and Figure 2.2 shows the remaining classes with their members, drawn in UML style. Table 2.2 summarises the purpose of every class.

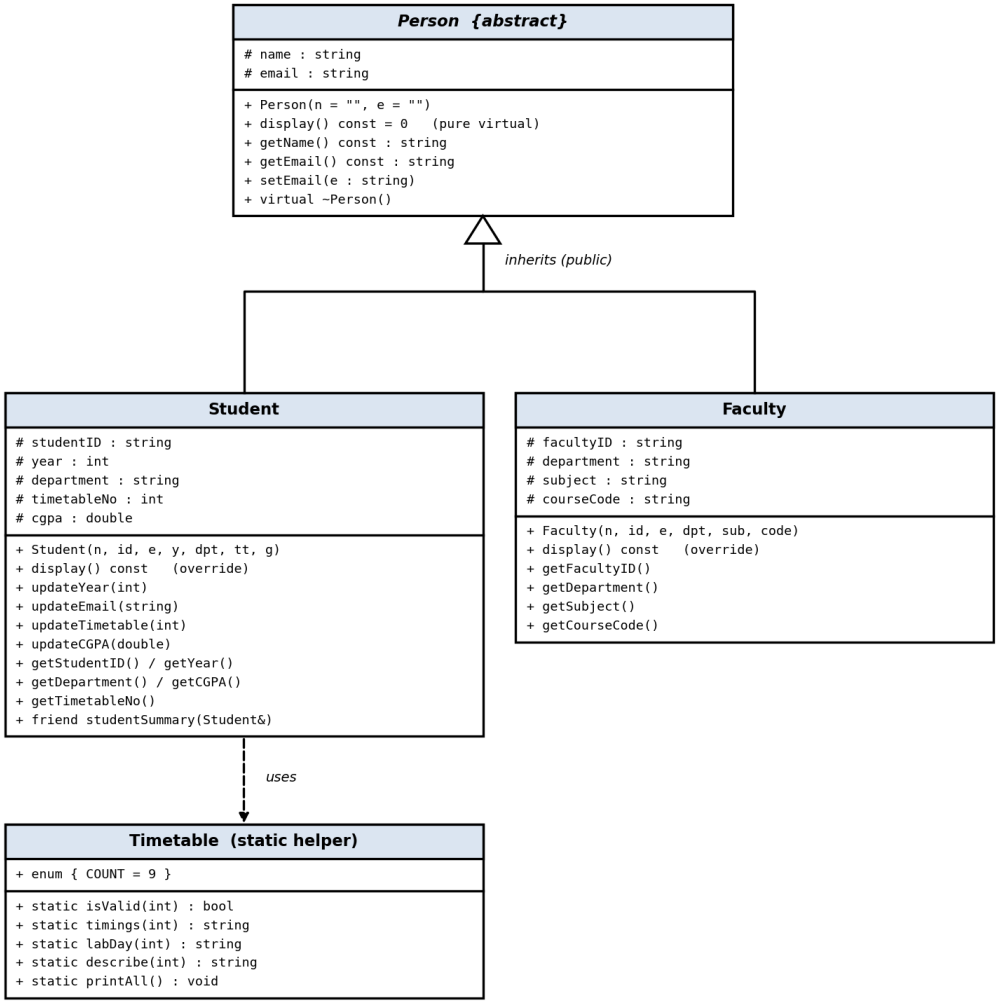


Figure 2.1: Class hierarchy: Person, Student, Faculty and Timetable

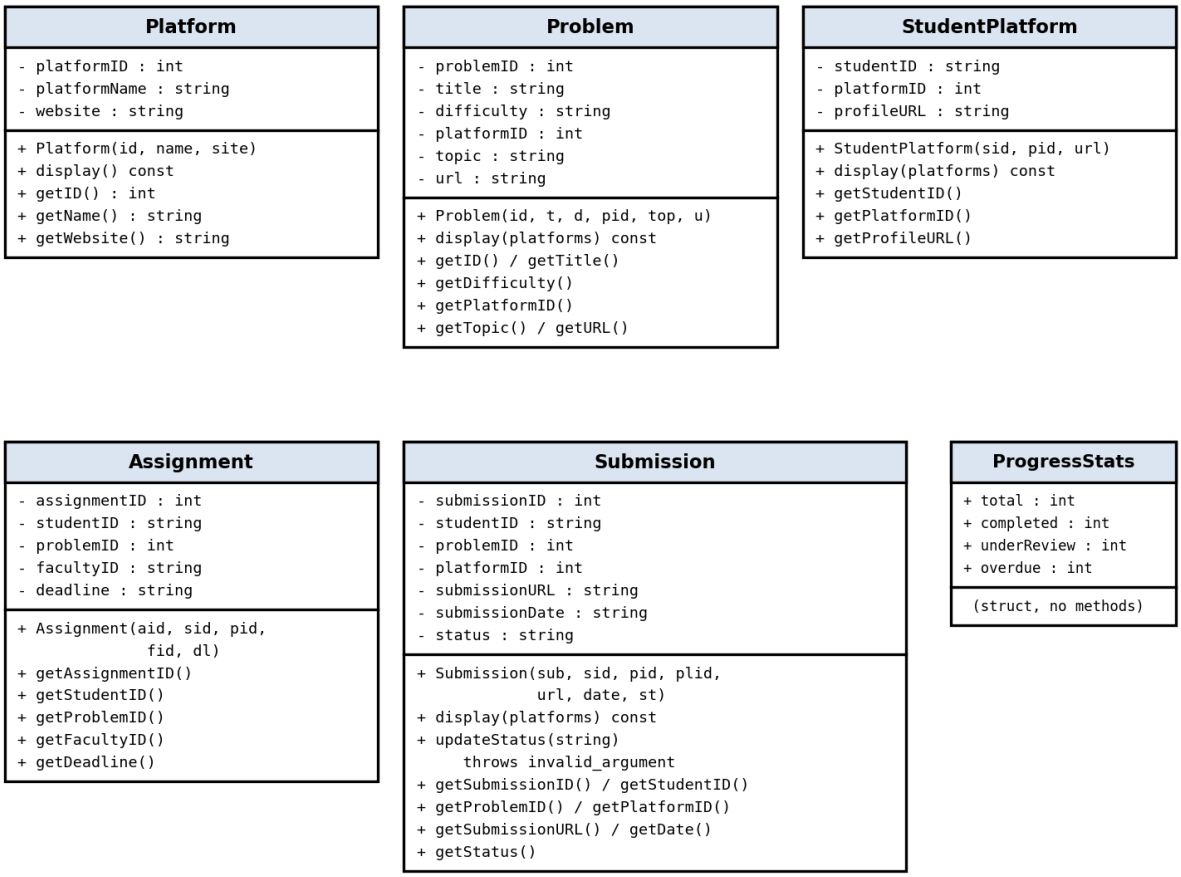


Figure 2.2: Platform, Problem, StudentPlatform, Assignment, Submission and ProgressStats

Table 2.2: Summary of the classes and the structure

Class	Purpose
`Person`	Abstract base class holding name and e-mail; declares the pure virtual `display()`.
`Student`	Student record: ID, year, department, timetable number and CGPA; validated update functions.
`Faculty`	Faculty record: ID, department, subject and course code.
`Timetable`	Static helper that validates and describes the nine university timetables.
`Platform`	Coding platform: ID, name and website, used to validate links.
`Problem`	Coding problem: ID, title, difficulty, platform ID, topic and link.
`StudentPlatform`	Link between a student and the profile address on a platform.
`Assignment`	A problem assigned to a student by a faculty member with a deadline.
`Submission`	A student's submission: link, date and status; `updateStatus()` validates the status.
`ProgressStats`	Structure with the counts total, completed, underReview and overdue.

The classes are related through stored identification numbers, not through pointers (Figure 2.3). For example, an `Assignment` stores a student ID, a problem ID and a faculty ID, and a `Submission` stores a student ID, a problem ID and a platform ID. The program uses these values to look the objects up in the corresponding vectors. The relationships are described below; those that the program enforces are stated as such.

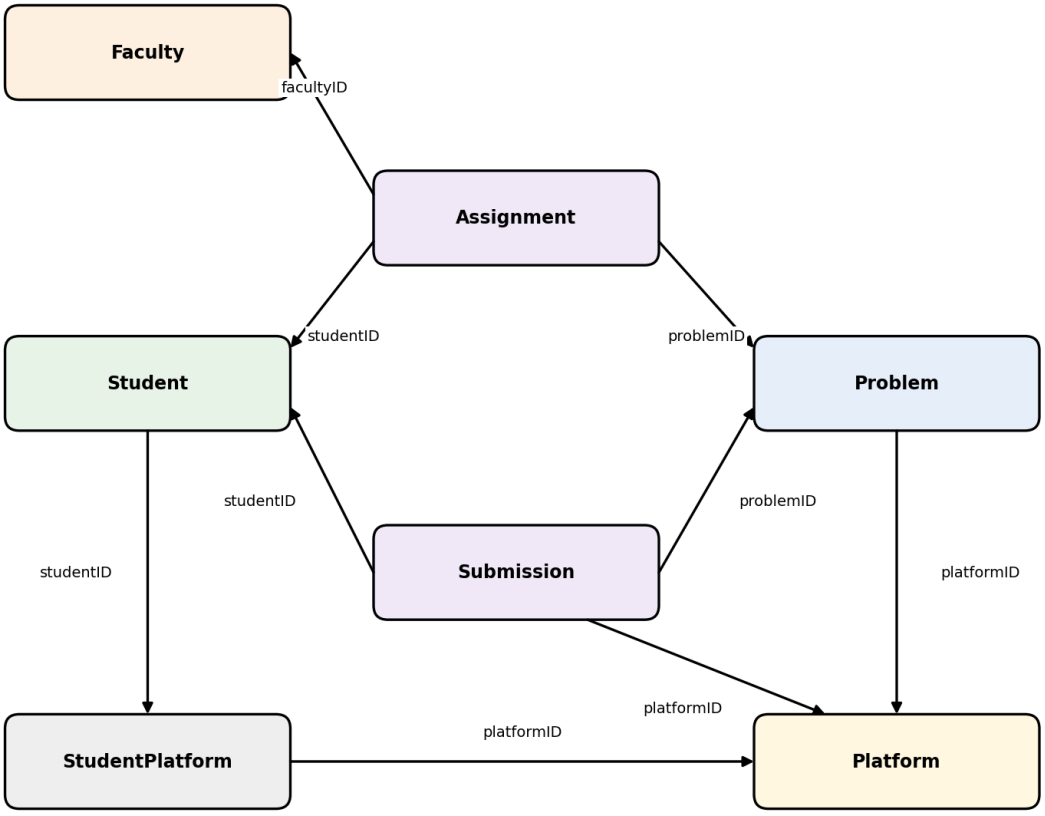


Figure 2.3: ID-based relationships between the principal classes

- **Problem and Platform:** a problem belongs to one platform. Enforced: `addProblem()` rejects an unknown platform ID and requires the link to begin with the platform's website.
- **StudentPlatform:** connects one student to one platform account. Enforced: a student can link a platform only once (`isStudentPlatformUnique()`).
- **Assignment:** connects a student, a problem and the assigning faculty member. Enforced: the student and problem must exist and the same problem cannot be assigned to the same student twice.
- **Submission:** belongs to a student and a problem. Enforced: `recordSubmission()` accepts it only if the problem was assigned to that student, and only the faculty member who assigned the problem can review it.

- **Deletion rules:** a platform cannot be deleted while problems or linked accounts use it; a problem cannot be deleted while it is assigned; a faculty member cannot be deleted while assignments exist for them. Deleting a student, after confirmation, also removes that student's assignments, submissions and linked accounts.
- **Faculty and Student:** these are not linked directly. They are connected only through assignments. This relationship is conceptual, not stored as a separate record.

2.3 Implementation Details

2.3.1 Start-up, Authentication and Main Menu

When the program starts, `main()` creates the vectors and loads the data files. If no records exist, `seedDefaultData()` adds sample data: five platforms, twenty students, ten or more faculty members, thirty-one problems, fifteen assignments, seven submissions and a few linked accounts. These are demonstration records supplied with the program. The main menu offers Admin Login, Faculty Login, Student Login, View Coding Platforms and Exit.

Authentication is simple and intended for demonstration. The administrator enters the user name `admin` with the password `admin123`. A faculty member enters a faculty ID that exists in the records and the password `faculty123`. A student enters a student ID that exists in the records and the password `student123`. These values are fixed in `main()` and the faculty and student passwords are shared by all users, so this mechanism does not provide real security and is listed under future scope.

2.3.2 Admin Module

The administrator manages people and platforms. The options of `adminMenu()` are listed in Table 2.3.

Table 2.3: Options of the Admin menu (adminMenu)

No.	Option	Function and behaviour
1	Add Student	`addStudent()`: unique ID, valid e-mail, year 1 to 4, department, timetable 1 to 9, CGPA 0.00 to 10.00.
2	View Students (ranked by CGPA)	`displayStudents()`: sorted table with rank.
3	View Students by Department	`displayStudentsByDepartment()`
4	View Students by Timetable	`displayStudentsByTimetable()`
5	Update Student	`updateStudent()`: year, e-mail, timetable or CGPA; exceptions are caught.
6	Delete Student	`deleteStudent()`: asks for confirmation and removes related records.
7	Search Student	`searchStudent()`
8	Sort Students	`sortStudents()`: by CGPA, name or department.
9	Add Faculty	`addFaculty()`
10	View Faculty	`displayFaculty()`
11	Delete Faculty	`deleteFaculty()`: refused if the faculty member has assignments.

No.	Option	Function and behaviour
12	View Platforms	`displayPlatforms()`
13	Delete Platform	`deletePlatform()`: refused if problems or linked accounts use the platform.
14	View Problems (oversight)	`displayProblems()`
15	View University Timetables	`Timetable::printAll()`
16	Logout	Returns to the main menu; data is then saved.

2.3.3 Faculty Module

The faculty menu (`facultyMenu()`) contains the functions for creating problems, assigning them and reviewing the work. A faculty member sees only submissions for problems that he or she assigned.

Table 2.4: Options of the Faculty menu (facultyMenu)

No.	Option	Function and behaviour
1	Add Coding Problem (with link)	`addProblem()`: title, difficulty (Easy, Medium, Hard), platform, topic and a link that must begin with the platform website; duplicate links are rejected.
2	View All Problems	`displayProblems()`
3	View Problems by Platform	`displayProblemsByPlatform()`
4	Search Problems	`searchProblemMenu()`: uses the overloaded `searchProblem()`.
5	Delete Problem	`deleteProblem()`: refused if the problem is assigned.
6	View Platforms	`displayPlatforms()`
7	Add Platform	`addPlatform()`: name and website.
8	Assign Problem to a Student	`assignProblem()`: deadline in YYYY-MM-DD format, not in the past.
9	Assign Problem to a Whole Department	`assignProblemToDepartment()`: skips students who already have the problem.
10	View My Assignments	`displayAssignmentsForFaculty()`
11	View Submissions Waiting for Review	`displaySubmissionsForFaculty()` for Pending and In Progress.
12	View All Submissions for My Assignments	`displaySubmissionsForFaculty()`
13	Review a Submission	`reviewSubmission()`: sets Pending, In Progress, Accepted or Rejected.
14	View Student Progress	`studentProgress()` for a student ID.
15	Generate Report	`generateReport()`: writes `student_report.txt`.
16	Logout	Returns to the main menu; data is then saved.

2.3.4 Student Module

A student can work only with his or her own records. After logging in with the student ID, the menu of `studentMenu()` is available (Table 2.5).

Table 2.5: Options of the Student menu (studentMenu)

No.	Option	Function and behaviour
1	View My Profile	`display()` of the student and the CGPA rank using `getStudentRank()`.
2	View My Linked Platforms	`displayStudentPlatforms()`
3	Link a New Platform Account	`addStudentPlatform()`: profile address must begin with the platform website; one account per platform.
4	View My Assigned Problems	`displayAssignmentsForStudent()` with the assignment state.
5	Search Problems	`searchProblemMenu()`
6	Record a Submission	`recordSubmission()`: only for an assigned problem that is not yet Accepted; status starts as Pending.
7	View My Submissions	`displaySubmissionsForStudent()`
8	View My Progress	`studentSummary()` (friend function) and `studentProgress()`.
9	Logout	Returns to the main menu; data is then saved.

2.3.5 Supported Coding Platforms

The platforms of Table 1.1 are stored as `Platform` objects. A problem refers to its platform by ID, and the platform website is used by `getValidURL()` to check that links for problems, submissions and profiles belong to the chosen platform. The application stores only the link and the status entered by users. It does not call platform APIs, does not check on the platform whether a problem was solved, and does not collect live statistics.

2.3.6 Input Validation

All keyboard input passes through helper functions that repeat the question until the value is acceptable (Table 2.6). For example, `getInteger()` clears the stream and asks again if non-numeric text is typed.

Table 2.6: Validation functions

Function	Check performed
`getInteger()`, `getIntegerInRange()`	Accepts only whole numbers, optionally within a range such as 1 to 4.
`getDoubleInRange()`	Accepts a decimal number within a range, used for CGPA 0.00 to 10.00.
`getNonEmptyString()`	Rejects empty text.
`isValidEmail()`, `getValidEmail()`	Requires an @ sign that is not first, a dot after it, no space, and a dot that is not last.
`isValidDate()`, `getValidDate()`	Requires the format YYYY-MM-DD with a month from 1 to 12 and a day from 1 to 31; deadlines cannot be in the past.
`isValidURL()`, `getValidURL()`	Requires http:// or https://, no spaces, and optionally the website prefix of the platform; "cancel" abandons the entry.
`isStudentIDUnique()`, `isFacultyIDUnique()`, `problemURLExists()`, `platformNameExists()`	Prevent duplicate IDs, problem links and platform names.
`confirmAction()`	Asks for confirmation before a student is deleted.

The date check verifies the pattern and simple ranges only; it does not test the number of days in each month.

2.3.7 Ranking, Searching and Sorting

Students are kept sorted by CGPA with `sort()` and `compareStudentByCGPA()`. `sortStudents()` also sorts by name or by department. Students with the same CGPA share the same rank, as calculated by `getStudentRank()`. Students are searched by ID (`findStudentIndex()`), and problems are searched by title or topic keyword (case-insensitive, through `containsIgnoreCase()`) or by ID. The listing functions copy the vector before filtering so the stored order is not disturbed.

2.3.8 Assignment States and Progress

The state of an assignment is not stored. It is calculated each time from the submissions and the deadline by `assignmentState()` (Table 2.7). `calculateProgress()` counts the states for one student, and the percentage is the number of Completed assignments divided by the number assigned.

Table 2.7: Assignment states and the progress counters

State	Condition (checked in this order)	Progress counter
Completed	The student has a submission with status Accepted for the problem.	completed
Under Review	A submission exists with status Pending or In Progress.	underReview
OVERDUE	No such submission and the deadline is earlier than today's date.	overdue
Not Completed	None of the above.	counted only in total

2.3.9 File Handling and Data Files

Data is saved by `saveAllData()` when a role logs out and when the program exits, and loaded by the `load...()` functions at start-up. Table 2.8 lists the files. Each line holds one record with fields separated by "|". Because a field containing this character would break the format, values should not include it.

Table 2.8: Files created and used by the program

File	Class	Contents of each line
`students.txt`	`Student`	ID, name, e-mail, year, department, timetable, CGPA
`faculty.txt`	`Faculty`	ID, name, e-mail, department, subject, course code
`platforms.txt`	`Platform`	ID, name, website
`problems.txt`	`Problem`	ID, title, difficulty, platform ID, topic, link
`studentplatforms.txt`	`StudentPlatform`	Student ID, platform ID, profile link
`assignments.txt`	`Assignment`	ID, student ID, problem ID, faculty ID, deadline
`submissions.txt`	`Submission`	ID, student ID, problem ID, platform ID, link, date, status

File	Class	Contents of each line
`student_report.txt`	(report output)	Rank, student details, assigned / accepted / under review / overdue counts and progress for every student

2.3.10 Limitations of the Present Implementation

- Authentication uses fixed passwords that are shared by all faculty and by all students; there are no individual accounts or encryption.
- Data is stored in plain text files, not in a database, and the field separator "|" must not appear in the data.
- There is no `Admin` class, no faculty-update function and no administrator function for adding problems.
- Submission links are checked only for format and website prefix; the program cannot confirm that the solution exists or is correct.
- `Person` pointers are not used, so run-time polymorphism is not exercised.
- The program has a console interface only.

2.4 Code

This section presents the parts of the program that carry the main workflow. Shorter excerpts that illustrate individual OOP concepts appear in Section 2.1. The complete source file, `CPP_CAPSTONE_PROJECT.txt` (2,396 lines), is submitted with the report.

2.4.1 Login and Main Menu (main)

After loading the data, `main()` shows the menu in a loop. The excerpt shows the administrator branch; the faculty and student branches follow the same pattern with an ID check.

```
        int choice = getInteger("\nEnter choice: ");

        if (choice == 1)
        {
            string username = getString("Username: ");
            string password = getString("Password: ");
            if (username == "admin" && password == "admin123")
            {
                adminMenu(students, faculty, problems, platforms, assignments, submissions, links);
                saveAllData(students, faculty, platforms, problems, assignments, submissions,
links);
            }
            else
                cout << "\nInvalid admin login." << endl;
        }
```

The role menu runs inside the `if` block. When it returns (Logout), `saveAllData()` writes all files, and control returns to the main menu loop.

2.4.2 Recording a Submission (recordSubmission)

This function enforces the rules for submissions: the problem must be assigned to the student, must not already be Accepted, and the link must belong to the problem's platform. The status always starts as Pending.

```
void recordSubmission(vector<Submission> &submissions,
                      const vector<Assignment> &assignments,
                      const vector<Problem> &problems,
                      const vector<Platform> &platforms,
                      const string &studentID)
{
    int problemID = getInteger("Enter the problem ID you solved: ");

    // A submission must correspond to a real assignment.
    if (!hasAssignment(assignments, studentID, problemID))
    {
        cout << "\nThis problem was not assigned to you. Cannot record submission." << endl;
        return;
    }
    // ...
    if (hasAcceptedSubmission(submissions, studentID, problemID))
    {
        cout << "\nThis problem is already marked Accepted. Nothing more to submit." << endl;
        return;
    }
    // ...
    string url = getValidURL("Enter your submission URL (or 'cancel'): ", site);
    if (url.empty()) { cout << "\nCancelled." << endl; return; }

    // Students cannot mark their own work as Accepted - it starts as Pending
    // and only the faculty who assigned the problem can review it.
    int id = nextSubmissionID(submissions);
    submissions.push_back(Submission(id, studentID, problemID, platformID,
                                     url, todayDate(), "Pending"));
    cout << "\nSubmission recorded (ID " << id << "). Status: Pending faculty review." << endl;
}
```

2.4.3 Assignment State and Progress (assignmentState, calculateProgress)

These two functions compute the progress shown to students and faculty. The state is derived from the submissions and the deadline, then counted.

```
string assignmentState(const vector<Submission> &submissions, const Assignment &a)
{
    if (hasAcceptedSubmission(submissions, a.getStudentID(), a.getProblemID()))
        return "Completed";
    if (hasUnderReviewSubmission(submissions, a.getStudentID(), a.getProblemID()))
        return "Under Review";
    if (a.getDeadline() < todayDate())
        return "OVERDUE";
    return "Not Completed";
}

struct ProgressStats
{
    int total;
    int completed;
    int underReview;
    int overdue;
};

ProgressStats calculateProgress(const vector<Assignment> &assignments,
                                const vector<Submission> &submissions,
                                const string &studentID)
{
    ProgressStats s;
```

```
    s.total = 0;
    s.completed = 0;
    s.underReview = 0;
    s.overdue = 0;

    for (int i = 0; i < (int)assignments.size(); i++)
    {
        if (assignments[i].getStudentID() != studentID) continue;

        s.total++;
        string state = assignmentState(submissions, assignments[i]);
        if (state == "Completed") s.completed++;
        else if (state == "Under Review") s.underReview++;
        else if (state == "OVERDUE") s.overdue++;
    }
    return s;
}
```

2.4.4 Protecting Related Records (deletePlatform)

Before a platform is erased, the function checks the problems and linked accounts that refer to it. This is the way the program keeps the ID-based relationships consistent.

```
void deletePlatform(vector<Platform> &platforms, const vector<Problem> &problems,
                    const vector<StudentPlatform> &links)
{
    int id = getInteger("Enter platform ID: ");
    int idx = findPlatformIndex(platforms, id);
    if (idx == -1) { cout << "\nPlatform not found." << endl; return; }

    for (int i = 0; i < (int)problems.size(); i++)
    {
        if (problems[i].getPlatformID() == id)
        {
            cout << "\nCannot delete: problems still belong to this platform." << endl;
            return;
        }
    }
    for (int i = 0; i < (int)links.size(); i++)
    {
        if (links[i].getPlatformID() == id)
        {
            cout << "\nCannot delete: students have linked accounts on this platform." << endl;
            return;
        }
    }

    platforms.erase(platforms.begin() + idx);
    cout << "\nPlatform deleted successfully." << endl;
}



// ============================================================
// STUDENT OPERATIONS  (CGPA ranking, timetable, CRUD)
// ============================================================
```

2.4.5 Report Generation (generateReport)

The report is written to `student_report.txt` using `ofstream`, from a sorted copy of the student vector.

```
void generateReport(const vector<Student> &students,
                    const vector<Assignment> &assignments,
                    const vector<Submission> &submissions)
{
    ofstream file("student_report.txt");
```

```
    if (!file) { cout << "\nUnable to create report." << endl; return; }

    vector<Student> ranked = students;
    sort(ranked.begin(), ranked.end(), compareStudentByCGPA);

    file << fixed << setprecision(2);
    file << "CODING PLATFORM - STUDENT REPORT (ranked by CGPA)\n";
    file << "Generated on: " << todayDate() << "\n";
    file << "================================================\n\n";

    for (int i = 0; i < (int)ranked.size(); i++)
    {
        ProgressStats s = calculateProgress(assignments, submissions, ranked[i].getStudentID());
    // ...
        file << "Rank        : " << (getStudentRank(ranked, ranked[i].getStudentID())) << "\n";
        file << "Student     : " << ranked[i].getName() << "\n";
        file << "ID          : " << ranked[i].getStudentID() << "\n";
        file << "Department  : " << ranked[i].getDepartment()
             << " (Year " << ranked[i].getYear() << ")\n";
        file << "CGPA        : " << ranked[i].getCGPA() << "\n";
        file << "Timetable   : " << Timetable::describe(ranked[i].getTimetableNo()) << "\n";
```

2.5 Output Screens

The following outputs were captured by compiling and running the submitted source file and using the default sample data created by `seedDefaultData()`. The characters typed by the user (menu numbers, user names and passwords) do not appear in the captured text because the console input is not echoed when it is supplied from a file. The text of each figure is reproduced exactly as printed by the program, trimmed only to the relevant lines. The team may replace these figures with its own screenshots of the same screens.

```
==========================================
 CODING PLATFORM / STUDENT PROGRESS
==========================================
1. Admin Login
2. Faculty Login
3. Student Login
4. View Coding Platforms
5. Exit
Enter choice: 1
Username: admin
Password: admin123
```

Figure 2.4: Main menu and rejected administrator login (Admin Login with an incorrect password)

```
===== STUDENT RANKING (by CGPA) =====
Rank  Student ID   Name                 Dept   Year  CGPA   Timetable
-------------------------------------------------------------------
1     25B11CS504   Meera Joshi          CSE    1     9.60   1
2     25B11CS502   Priya Reddy          CSE    2     9.35   5
3     25B11IT703   Ishita Bose          IT     4     9.25   9
4     25B11AI586   Jahnavi              AIML   2     9.12   5
5     25B11IT701   Divya Nair           IT     2     9.05   6
6     25B11AI587   Sneha Iyer           AIML   2     8.90   5
7     25B11CS506   Ritika Agarwal       CSE    3     8.85   8
8     25B11EC601   Ananya Gupta         ECE    2     8.72   3
9     25B11AI589   Pooja Desai          AIML   3     8.60   8
10    25B11CS501   Rahul Sharma         CSE    2     8.45   6
11    25B11ME801   Kavya Pillai         ME     4     8.30   9
12    25B11AI588   Karthik Rao          AIML   1     8.15   1
13    25B11CS505   Siddharth Rao        CSE    4     8.05   9
14    25B11CS503   Arjun Verma          CSE    3     7.80   7
15    25B11IT702   Rohan Mehta          IT     1     7.60   2
16    25B11AI590   Suresh Babu          AIML   1     7.45   3
17    25B11EC602   Vikram Singh         ECE    3     7.35   8
18    25B11EC603   Naveen Chandra       ECE    1     7.10   2
19    25B11ME802   Aditya Kumar         ME     3     6.95   7
20    25B11ME803   Manoj Pillai         ME     2     6.70   4
```

Figure 2.5: Admin menu, option 2: students ranked by CGPA (first ten of twenty sample records)

```
--- Platforms ---
ID   Name             Website
-------------------------------------------------
1    LeetCode         https://leetcode.com
2    CodeChef         https://www.codechef.com
3    HackerRank       https://www.hackerrank.com
4    GeeksforGeeks    https://www.geeksforgeeks.org
5    Codeforces       https://codeforces.com
6    codeforges       https://codeforges.com
```

Figure 2.6: Faculty menu, option 6: supported coding platforms

```
====== STUDENT PROGRESS ======
Student ID     : 25B11AI586
Assigned       : 2
Accepted       : 1
Under Review   : 1
Overdue        : 0
Progress       : 50.00%
```

Figure 2.7: Faculty menu, option 14: progress of student 25B11AI586

```
Assignment ID : 1
Student ID    : 25B11AI586
Assigned By   : F101
Problem       : 101 - Two Sum [Easy, Arrays]
Platform      : LeetCode
Problem Link  : https://leetcode.com/problems/two-sum/
Deadline      : 2026-12-31
State         : Completed

Assignment ID : 2
Student ID    : 25B11AI586
Assigned By   : F101
Problem       : 103 - Valid Parentheses [Easy, Stacks]
Platform      : LeetCode
Problem Link  : https://leetcode.com/problems/valid-parentheses/
Deadline      : 2026-12-31
State         : Under Review
```

Figure 2.8: Student menu, option 4: assigned problems with assignment state (student 25B11AI586)

```
Student Summary
Name       : Jahnavi
Student ID : 25B11AI586
Department : AIML
Year       : 2
CGPA       : 9.12
Timetable  : 5

===== STUDENT PROGRESS =====
Student ID   : 25B11AI586
Assigned     : 2
Accepted     : 1
Under Review : 1
Overdue      : 0
Progress     : 50.00%
```

Figure 2.9: Student menu, option 8: student summary and progress

In Figure 2.7 and Figure 2.9 the student has two assignments: one Completed (an Accepted submission) and one Under Review (a Pending submission), which gives 1 of 2, or 50.00%, in accordance with the calculation in Section 2.3.8.

CHAPTER 3: CONCLUSION AND FUTURE SCOPE

3.1 Conclusion

The project delivers a working C++ console application that brings the scattered activity of coding practice into one system. Administrators maintain student, faculty and platform records; faculty add problems from five coding platforms, assign them to students or to whole departments with deadlines, and review submissions; and students link their profile addresses, record submissions and see their progress. Students are ranked by CGPA, and a ranked progress report can be written to a text file.

The implementation applies the main object-oriented concepts in a realistic setting: an abstract base class `Person` with the derived classes `Student` and `Faculty`, encapsulated record classes, a virtual `display()` function, overloaded search functions, a friend function, exception handling for invalid updates, STL vectors with `sort()`, and stream-based file handling. Input validation and relationship checks keep the records consistent: links must belong to the correct platform, submissions are accepted only for assigned problems, and only the assigning faculty member can review them.

The program has clear limits. It stores data in text files, uses fixed demonstration passwords, has no Admin or Progress class, and records only links and statuses entered by users; it does not communicate with the coding platforms. These items are the starting point for the improvements below.

3.2 Future Scope

The following improvements are suggestions and are not part of the completed implementation:

- Replace the shared passwords with individual accounts and stored (hashed) passwords.
- Store the data in a database instead of text files, which would also allow the entity relationships shown in the presentation to be implemented as tables.
- Introduce an `Admin` class and a `Progress` class, and refactor the menus so that polymorphic calls are made through `Person` pointers.
- Connect to the public interfaces of the coding platforms, where available, to confirm submissions and to retrieve statistics automatically.
- Add faculty update functions, calendar-accurate date validation and richer submission results such as the verdict and the number of attempts.
- Provide analytical reports and charts of department-wise and topic-wise progress.
- Build a graphical or web interface and add notifications for approaching deadlines.